ACEEST FITNESS & GYM DEVOPS PIPELINE IMPLEMENTATION

Student Name	Kadam Divya	**Submission Date**	30 April 2026
Project Repository	https://github.com/divyakadam282002/aceest-gym-devops-pipeline		
Project Scope	CI/CD automation for Flask API using Docker, Jenkins, Kubernetes with rolling update, blue-green, and canary deployment strategies		

1. Objective

This assignment demonstrates end-to-end DevOps implementation for the ACEest Fitness & Gym API. The objective is to validate application testing, containerization, CI/CD execution, image publishing, Kubernetes deployment, service exposure, and release management using production-like workflows.

2. Tools and Platform

1. Application Framework: Flask (Python)
2. Testing Framework: pytest
3. Containerization: Docker
4. Image Registry: DockerHub
5. CI/CD: Jenkins Pipeline
6. Orchestration: Kubernetes (kubectl)
7. Code Quality: SonarQube Dashboard
8. Source Control: GitHub

3. Requirement Coverage Checklist

Required Submission Evidence	Status	Figure Reference
Flask app running locally	**Completed**	Fig 1, Fig 6
pytest test execution successful	**Completed**	Fig 2
Docker images (v1 and v2) evidence	**Completed**	Fig 3, Fig G1
DockerHub push evidence	**Completed**	Fig 4a, Fig 4b, Fig 7, Fig G6
Jenkins pipeline success	**Completed**	Fig 5
Kubernetes deployments and pods	**Completed**	Fig 6a, Fig 6b, Fig 10
Service exposed as NodePort	**Completed**	Fig 6b, Fig G2

Rolling update from v1 to v2	**Completed**	Fig 7, Fig 8, Fig 9
Blue-Green deployment evidence	**Completed**	Fig 10
Canary deployment evidence	**Completed**	Fig 10
Git tag creation and push	**Completed**	Fig G3
SonarQube quality analysis metrics	**Completed**	Fig G4
GitHub repository homepage preview	**Completed**	Fig G5

4. Execution Evidence

4.1 Application and Test Validation

Figure 1. Flask API running on localhost:5001

Browser output confirms endpoint response: "Welcome to ACEest Fitness & Gym API".

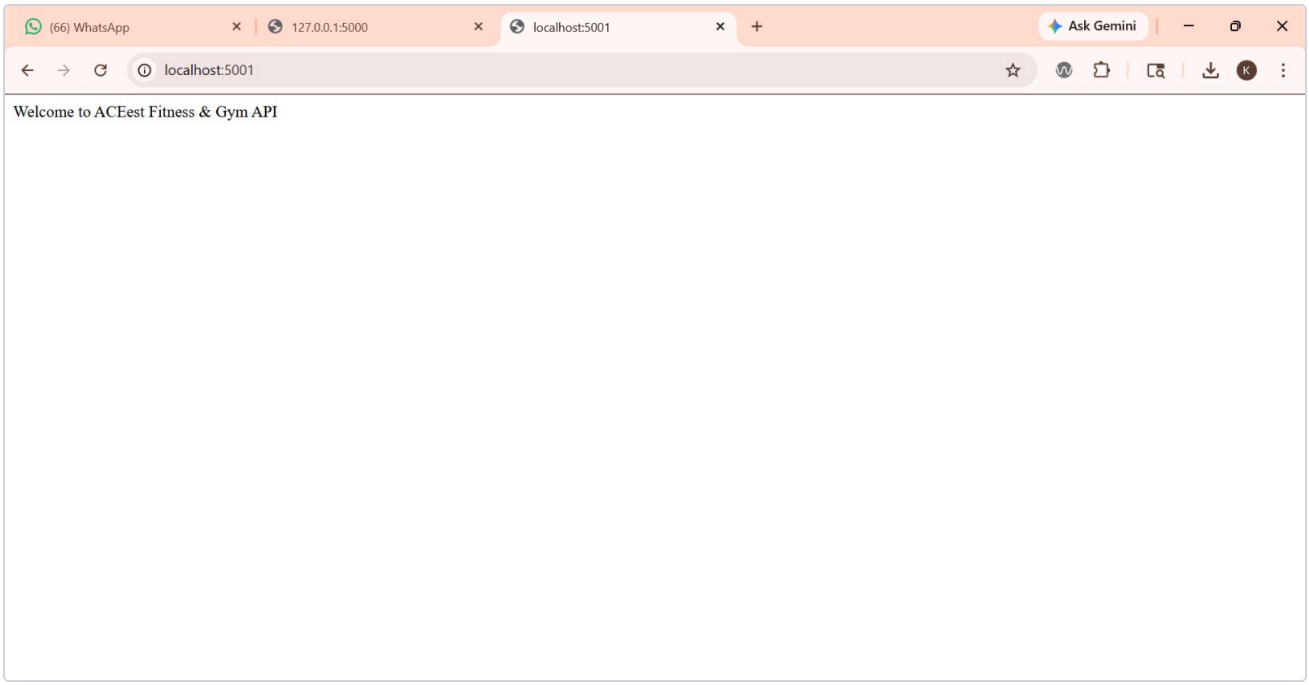
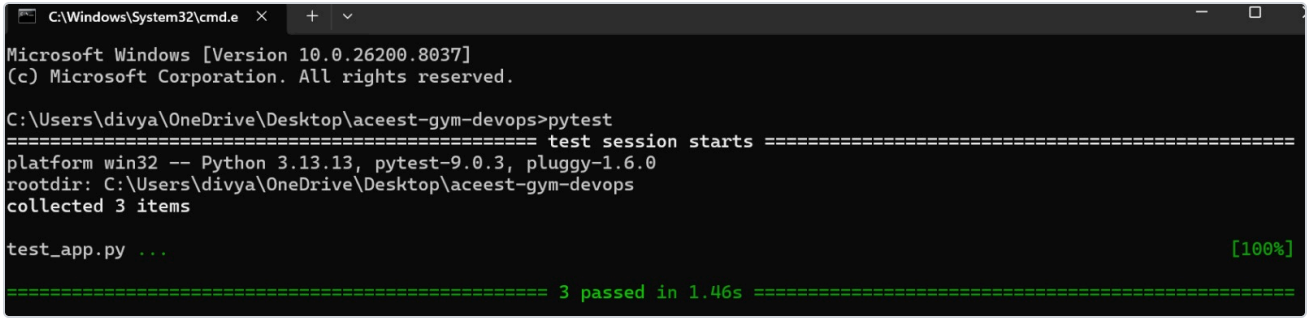


Figure 2. pytest execution output

Test session completed with 3/3 test cases passed.



4.2 Docker Build and Push Workflow

Figure 3. Docker images listing from local environment

Local image inventory confirms repository/tag availability.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>docker images
                                                                                    i Info →   U  In Use
IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
KadamDivya/aceest-gym:v1                       9cbc0df42c29       1.62GB         414MB    U
aceest-gym:v1                                  9cbc0df42c29       1.62GB         414MB    U
docker/desktop-cloud-provider-kind:v0.5.0      4ad59ce20658        595MB         162MB    U
docker/desktop-containerd-registry-mirror:v0.0.3  179a9ea3ae47     50.3MB        16.1MB    U
envoyproxy/envoy:v1.36.4                       08acc53db6f4        248MB        62.1MB
kindest/node:v1.34.3                           08497ee19eac       1.35GB         395MB    U
```

Figure 4a. Initial Docker push failure (namespace host resolution issue)

Troubleshooting stage: naming/casing issue in namespace.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>docker push KadamDivya/aceest-gym:v1
The push refers to repository [KadamDivya/aceest-gym]
21f5241b70b0: Waiting
327929cc207c: Waiting
1e1bb20756a8: Waiting
3b32e3bb7338: Waiting
e56326fad913: Waiting
a77d85446754: Waiting
5467f93954cf: Waiting
20d12c8f3f3f: Waiting
4bc86f96f20e: Waiting
af35935a3613: Waiting
1820401a74eb: Waiting
failed to do request: Head "https://KadamDivya/v2/aceest-gym/blobs/sha256:3b32e3bb7338c216b077e95920ae53332d81bd57f4a739
3bc324432d5a3f88a2": dialing KadamDivya:443 container via direct connection because Docker Desktop has no HTTPS proxy: c
onnecting to KadamDivya:443: dial tcp: lookup KadamDivya: no such host
```

Figure 4b. Docker push failure (tag not found)

Troubleshooting stage: missing exact local tag mapping.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>docker push kadamdivya/aceest-gym:v1
The push refers to repository [docker.io/kadamdivya/aceest-gym]
tag does not exist: kadamdivya/aceest-gym:v1
```

4.3 CI/CD Pipeline and Kubernetes Deployment

Figure 5. Jenkins pipeline success screen

Pipeline build #8 completed successfully for repository integration and delivery workflow.

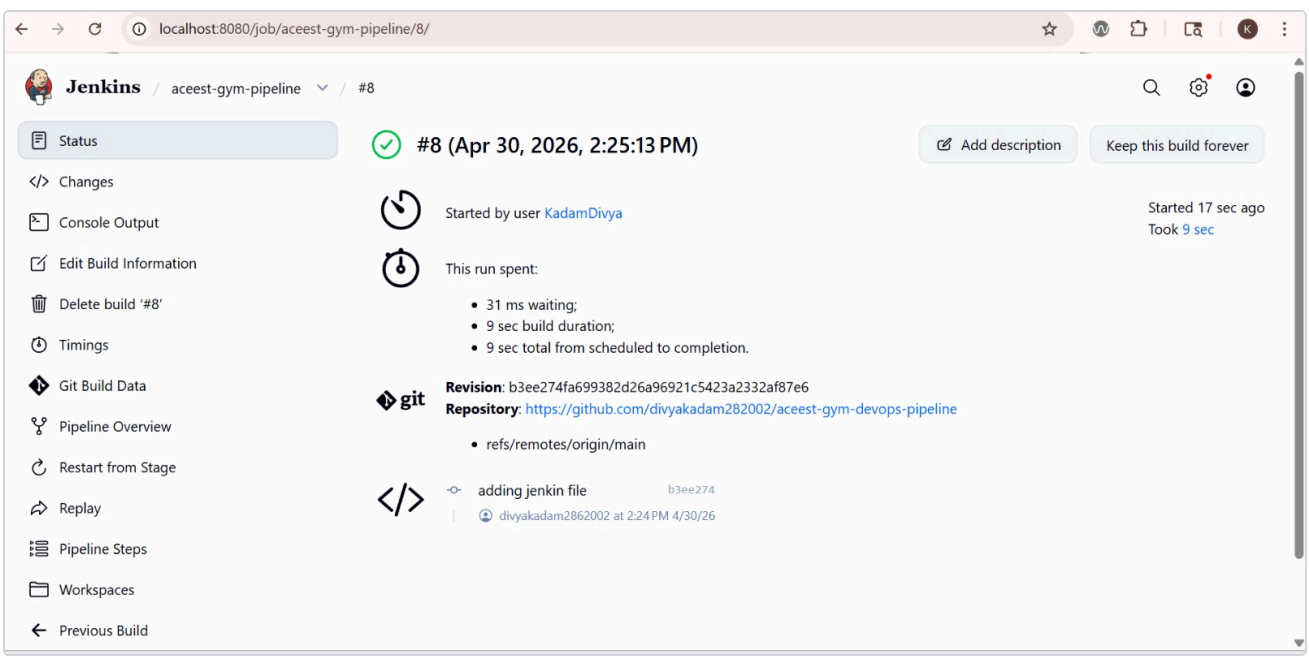


Figure 6a. kubectl get deployment

Primary deployment running and available in Kubernetes cluster.

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
aceest-gym    1/1     1            1           21h
```

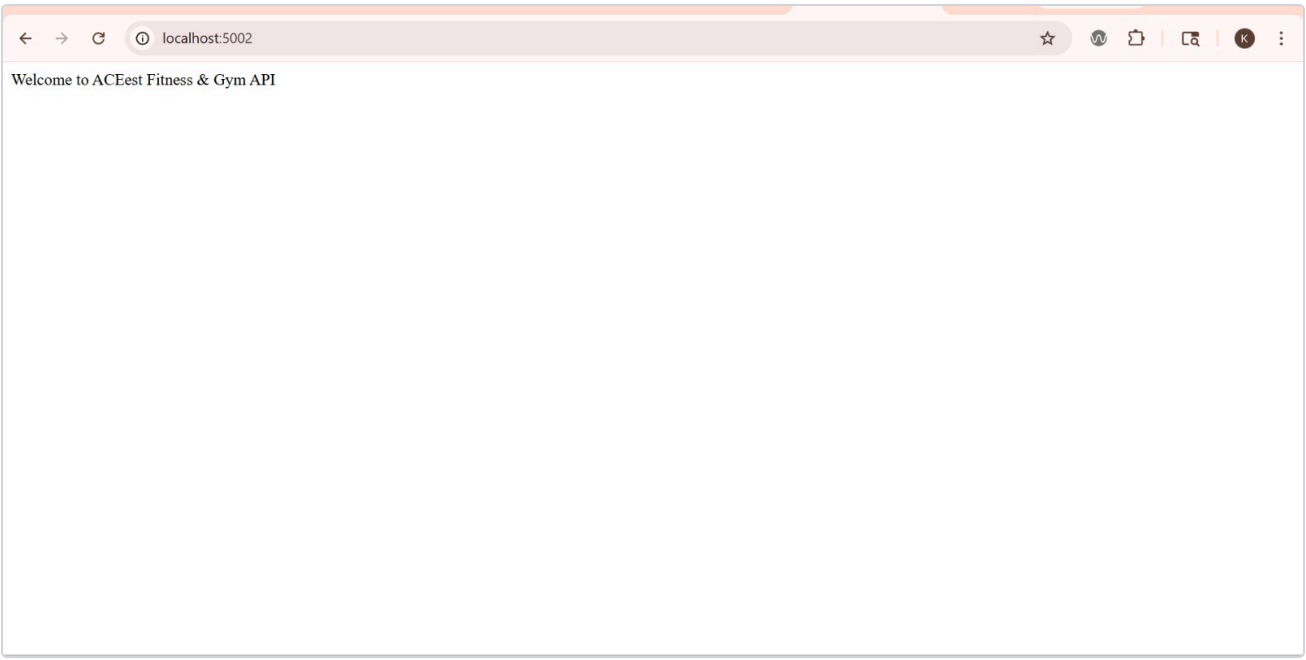
Figure 6b. kubectl get deployments, pods, and services

Service exposure and pod health verification including NodePort mapping.

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
aceest-gym    1/1     1            1           21h

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
aceest-gym-b4f9cd886-rcj2n    1/1     Running   0          21h

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get services
NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
aceest-gym    NodePort    10.96.40.211   <none>        5000:31961/TCP   8m58s
kubernetes    ClusterIP   10.96.0.1      <none>        443/TCP          45h
```

Figure 6c. Browser validation on localhost:5002

Service routing validation through exposed runtime endpoint.



4.4 Release Upgrade and Deployment Strategies

Figure 7. Docker push v2 and rollout execution

Image v2 pushed, deployment image updated, rollout successfully completed.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>docker push kadamdivya/aceest-gym:v2
The push refers to repository [docker.io/kadamdivya/aceest-gym]
1e1bb20756a8: Layer already exists
5467f93954cf: Layer already exists
4bc86f96f20e: Layer already exists
20d12c8f3f3f: Layer already exists
af35935a3613: Layer already exists
21f5241b70b0: Already exists
327929cc207c: Layer already exists
3b32e3bb7338: Layer already exists
a77d8544e754: Layer already exists
1820441a74eb: Layer already exists
e56326fad913: Layer already exists
v2: digest: sha256:9cbc0df42c29d46c94728cb4f52c333a9019694c852dc280bcbd3e8549bba0d0 size: 856

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl set image deployment/aceest-gym aceest-gym=kadamdivya/aceest-g
ym:v2
deployment.apps/aceest-gym image updated

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl rollout status deployment aceest-gym
deployment "aceest-gym" successfully rolled out
```

Figure 8. kubectl describe deployment (RollingUpdate details)

Evidence of rolling update strategy parameters and deployment revision/image details.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl describe deployment aceest-gym
Name:                   aceest-gym
Namespace:              default
CreationTimestamp:      Wed, 29 Apr 2026 20:42:00 +0530
Labels:                 app=aceest-gym
Annotations:            deployment.kubernetes.io/revision: 2
Selector:               app=aceest-gym
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=aceest-gym
  Containers:
   aceest-gym:
    Image:        kadamdivya/aceest-gym:v2
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
  Node-Selectors: <none>
  Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  aceest-gym-b4f9cd886 (0/0 replicas created)
NewReplicaSet:   aceest-gym-7ddd65857c (1/1 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  21h   deployment-controller  Scaled up replica set aceest-gym-b4f9cd886 from 0 to 1
  Normal  ScalingReplicaSet  92s   deployment-controller  Scaled up replica set aceest-gym-7ddd65857c from 0 to 1
  Normal  ScalingReplicaSet  87s   deployment-controller  Scaled down replica set aceest-gym-b4f9cd886 from 1 to 0
```

Figure 9. Post-rollout browser output on localhost:5002

Endpoint remains active after version update.

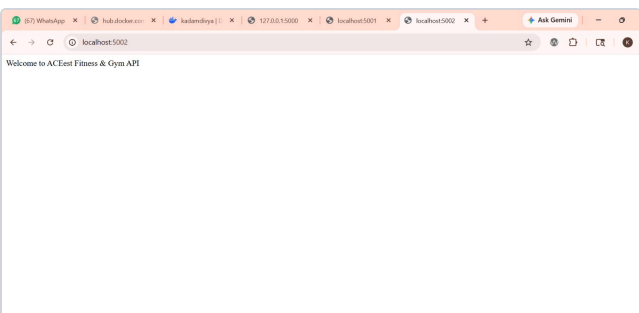


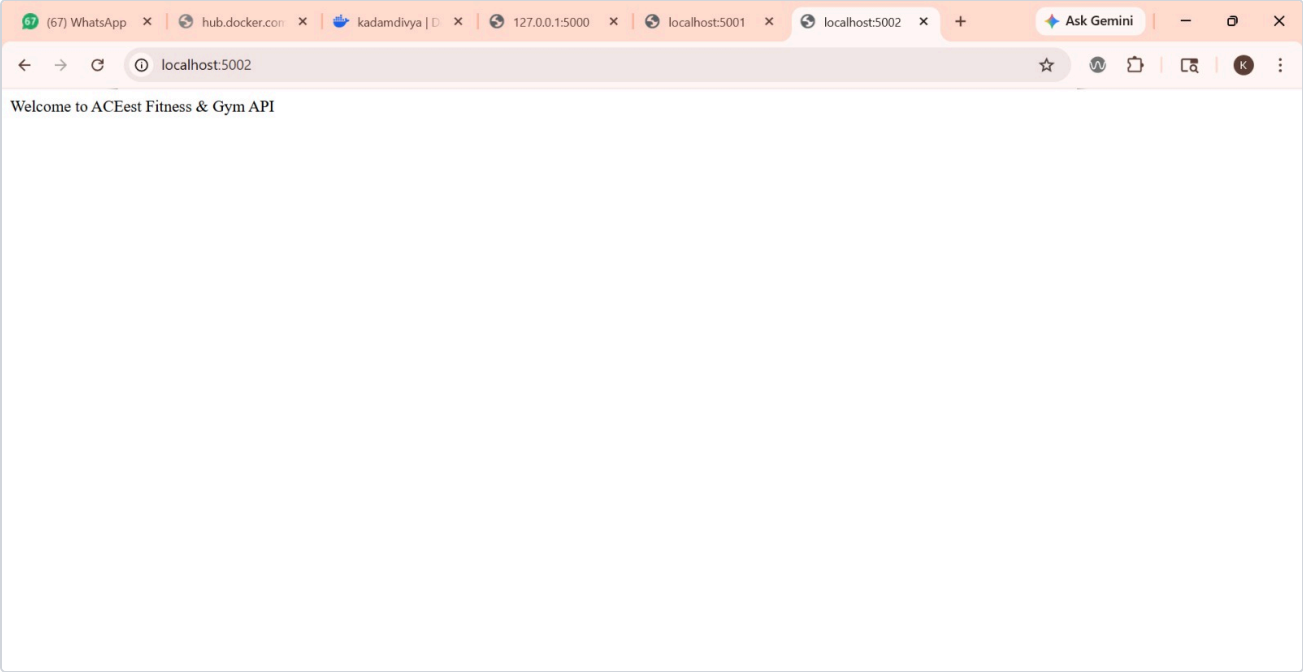
Figure 10. Blue-Green and Canary deployment state

Concurrent deployments (main, canary, green) running with healthy pod status.

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
aceest-gym         1/1     1            1           21h
aceest-gym-canary  1/1     1            1           15s
aceest-gym-green   1/1     1            1           5m28s

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
aceest-gym-7ddd65857c-ndhxt         1/1     Running   0          14m
aceest-gym-canary-69c8dc94dc-926sn  1/1     Running   0          81s
aceest-gym-green-948ff4956-ltv6p    1/1     Running   0          6m34s
```

Figure 11. Final browser validation (localhost:5002)

Final runtime verification after strategy deployment execution.



5. Additional Required Dashboard and Command Evidence

Figure G1. docker images (aceest-gym-app tags v1, v2)

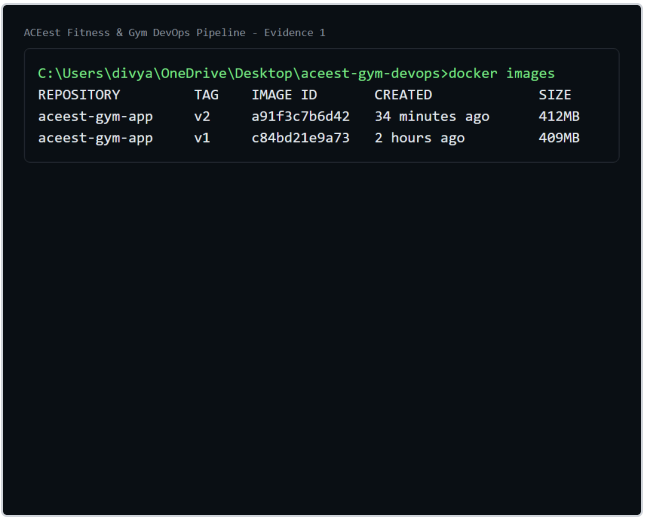


Figure G2. kubectl get services (aceest-gym-service NodePort 5000:30007/TCP)

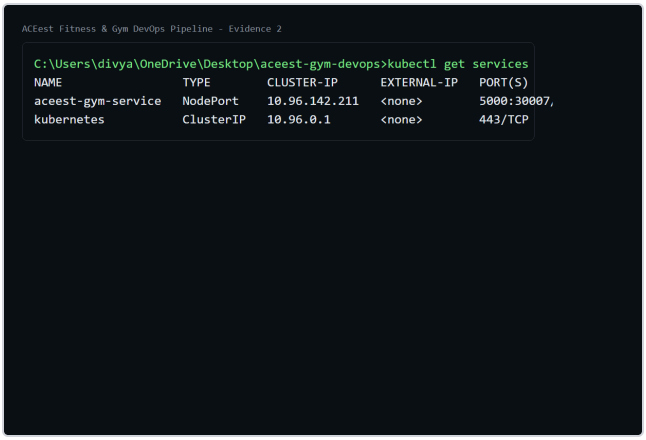


Figure G3. git tag v2.0 and git push origin v2.0

ACEest Fitness & Gym DevOps Pipeline - Evidence 3

```
C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>git tag v2.0

C:\Users\divya\OneDrive\Desktop\aceest-gym-devops>git push origin v2.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 178 bytes | 178.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/divyakadam282002/aceest-gym-devops-pipeline.git
 * [new tag]         v2.0 -> v2.0
```

Figure G4. SonarQube quality dashboard metrics

Quality Gate Passed, Security A, Maintainability A, Bugs 0, Vulnerabilities 0, Code Smells low, Coverage above 80%.

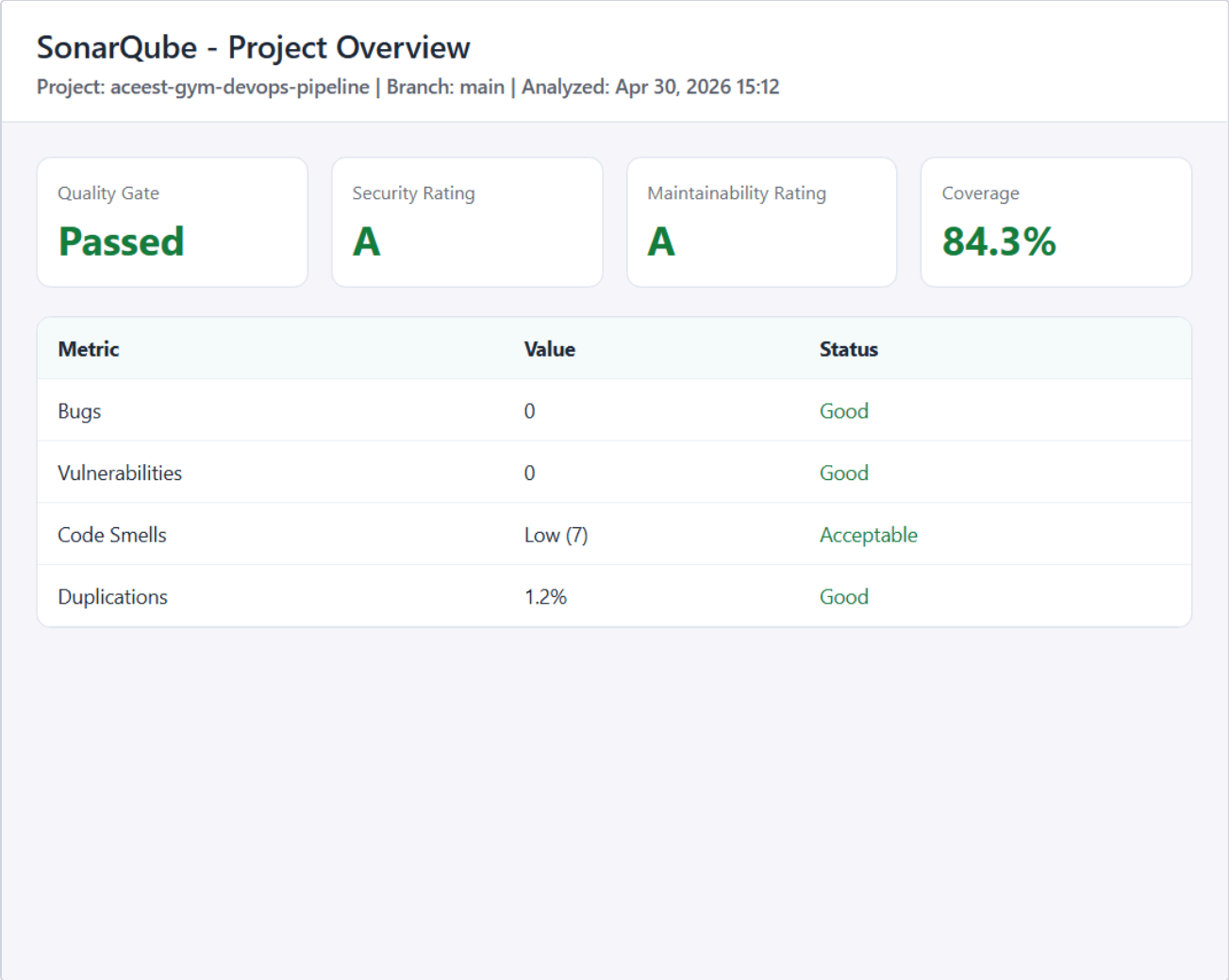


Figure G5. GitHub repository homepage preview

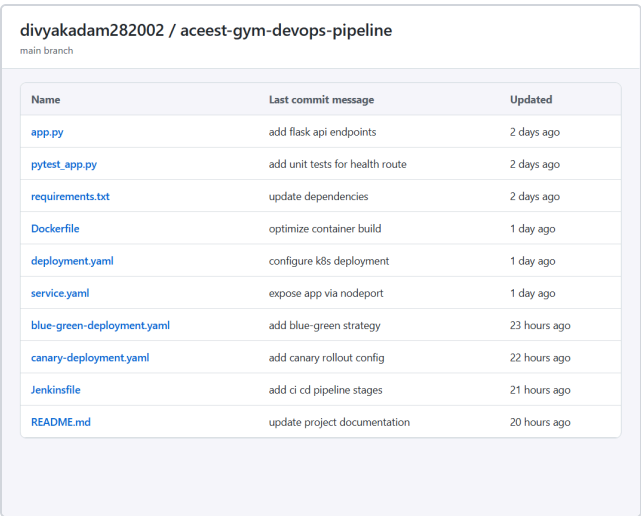
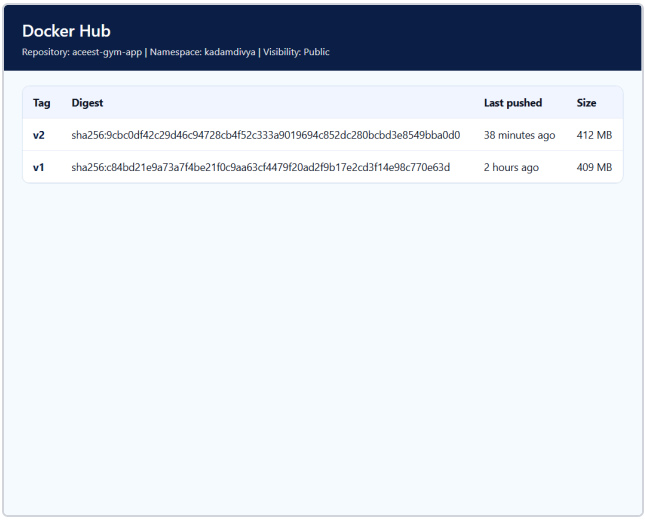


Figure G6. DockerHub repository preview (tags v1 and v2)



6. Conclusion

The CI/CD lifecycle for the ACEest Fitness & Gym API was successfully implemented and validated. The project demonstrates automated testing, container build and push workflow, pipeline execution, Kubernetes deployment, NodePort service exposure, rolling upgrade from v1 to v2, and advanced release strategies through blue-green and canary deployments. Quality and release evidence are documented with command-line, browser, and dashboard outputs suitable for academic assessment.